jigsaw-agile-community-rules

[8th] Qwen3-14B*3 + Llama2-13B*1 + bge-base-en-v1.5

Rank	8
Team	Reddit comment moderators
Author	muyouqian4
Topic ID	613195
Version	Original

Source: https://www.kaggle.com/competitions/jigsaw-agile-community-rules/discussion/613195
Retrieved with Kaggle CLI on 2026-10-21.

Wow! Huge thanks to the competition host Jigsaw/Conversation AI and the organizers for this competition. Before participating, I knew almost nothing about NLP, and I learned so many valuable lessons from it.

Huge thanks to the Kaggle community for its great discussion atmosphere and to the selfless Kagglers who open-source their work.

Huge thanks to my teammates @williamwu88, @justforfun44 for their valuable suggestion and for sharing experience and effort! Looking forward to a pleasant collaboration next time!

All content below describes the training and inference performed during the testing phase.

Without access to the complete rules and data, online learning became the consensus in the Jigsaw community. Therefore, we believed the key was how to ensemble larger-scale and a greater number of LLM models within the given time limits and memory constraints.

In Kaggle's "harsh" environment, we achieved efficient training and inference of 3 Qwen3-14B, 1 Llama2-13B, and 2 Llama3.1-8B models on T4*2 within 12 hours(QLora), based on unsloth (https://huggingface.co/unsloth). No CV, code was only adjusted based on the LB. Each submission was an ensemble of the same model and parameters with 4 different seeds, ensuring stable improvement rather than noise. The LB is a random split. 29% of 55k data is not a small amount. The mAUC evaluation metric is quite stable, so we believed the LB was trustworthy.

This is the model composition for the two final submissions:

LB=0.933,PB=0.927

seed	model
42	qwen3-14b
43	qwen3-14b
44	qwen3-14b
45	llama2-13b
46	bge-base-en-v1.5

LB=0.932,PB=0.926

seed	model
42	qwen3-14b
43	qwen3-14b
44	qwen3-14b
45	llama2-13b

seed	model
46	llama3.1-8b
47	llama3.1-8b

1. Adding a Classification Head:

1. Unsloth's official text classification implementation is via SFT (https://colab.research.google.com/github/timothelaborie/text_classification_scripts/blob/main/unsloth_classification.ipynb). We modified it to a standard AutoModelForSequenceClassification head, simply because it's more conventional. We are not sure which one is better.

2. Parallel Training, Fully Utilizing T4*2:

We couldn't find official unsloth code for parallel training, so we modified and adapted standard parallel training code. There were some pitfalls [?]. If you want to adapt this to other tasks, please note:

1. The QWEN3-14B model can only train two models in parallel and cannot use the `batch*2` training like the 8B model. The 8B model has no such restriction. We guess this is because the model must first be loaded onto GPU:0 simultaneously, and only if there is no OOM, it is then distributed to the two GPUs. We couldn't solve this issue. Llama2-13B and other smaller models can be trained in parallel.
2. Unsloth must be initialized before training.

3. Training Parameter Optimization:

Mainly `group_by_length=True`, which sped up training by 2x.

4. Dataset

1. For each `(body, rule)` combination, find the most frequent label and deduplicate the samples.
2. Found the official unlabeled dataset. We couldn't know the positive sample criteria for the Rules, but we could identify absolute negative samples and add them to the training set.
3. Found the official unlabeled dataset. Tried to add guessed positive samples based on the Rules to the training set. If the LB dropped, we reverted it.

5. Inference

1. Evenly distribute data to the two GPUs, sort the data from shortest to longest, and perform parallel inference. This sped it up by 4x.
2. Note: GPU:1 must wait a few seconds to avoid conflicts.
3. Split the data into 3 buckets and specified different `batch_size` for inference in each, reducing inference time and allowing us to add one more 8B model.

6. Ensemble

1. Ensembling based on global ranking was better than ensembling based on within-rule ranking.

7. What Didn't Work for Us

1. Pseudo-labeling
2. Simulating CV during training

8. Looking forward to the selfless sharing from other kagglers!

哇！非常感谢主办方Competition Host Jigsaw/Conversation AI和主持人举办的竞赛，在参加前，我对NLP几乎一无所知，我从中学习到了很多宝贵的经验。

非常感谢kaggle的良好的讨论社区氛围和无私开源的kaggler。

非常感谢我的队友@williamwu88, @justforfun44的valuable suggestion and for sharing experience and effort!期待下次的愉快合作！

以下所有内容都是在测试时进行的训练和推理。

在无法获得全部Rule和数据的情况下，在线学习成为了Jigsaw社区的共识。因此，我们认为关键在于，如何在规定的时间限制和显存限制下集成规模更大和数量更多的LLM模型。

在kaggle的“严苛”环境下，我们实现了12小时内的在T4*2上基于unsloth (https://huggingface.co/unsloth) 的3个Qwen3-14B,1个Llama2-13B和2个Llama3.1-8B的高效训练和推理(QLora)。没有CV，代码只根据LB调整，每次提交4个不同种子的相同模型和参数的集成，确保稳定提升而非噪声。LB是随机划分，55k的29%数据量不算小，mAUC的评估指标比较稳定，所以我们认为是可以相信LB的。

这是最终提交的两个代码的模型组成:

LB=0.933,PB=0.927

seed	model
42	qwen3-14b
43	qwen3-14b
44	qwen3-14b
45	llama2-13b
46	bge-base-en-v1.5

LB=0.932,PB=0.926

seed	model
42	qwen3-14b
43	qwen3-14b
44	qwen3-14b
45	llama2-13b
46	llama3.1-8b
47	llama3.1-8b

1.增加分类头:

1.Unsloth的官方的文本分类的实现是通过SFT (https://colab.research.google.com/github/timothelaborie/text_classification_scripts/blob/main/unsloth_classification.ipynb) ，我们修改成了标准的AutoModelForSequenceClassification的分类头，只是更符合习惯，不确定哪个更好。

2.并行训练，充分利用T4*2：

我们没有找到unsloth官方的并行训练的相关代码，所以是基于常规的并行训练代码进行的修改和适配，有一些坑，如果想修改至其他任务，请注意:

1.QWEN3-14B模型只能并行训练两个模型而无法使用8B模型的batch*2的训练，但8B模型没有限制，猜测是模型必须同时先加载到GPU:0上，如果没有OOM，然后才分配到两个GPU上。我们无法解决该问题。llama2-13B和其他更小的模型可以并行训练。

2.Unsloth的初始化必须在训练前。

3.训练参数优化：

主要是group_by_length=True，加速了2倍

4.数据集

1.找出每个 (body, rule) 组合中，出现次数最多的那个标签，去重样本

2.找到官方的未标注的数据集，我们无法知道Rule的正样本标准，但是能找出绝对的负样本，加入训练集

3.找到官方的未标注的数据集，尝试根据Rule加入猜测的正样本，加入训练集，如果LB下降则回退

5.推理

1.均匀分配数据给两个GPU，从短至长排序数据，并行推理，加速了4倍

2.注意GPU:1必须等待几秒避免冲突

3.将数据拆分为3个桶并分别指定batch_size推理，减少推理时间，让我们能再加入一个8B模型

6.集成

1.全局排名的集成优于规则内排名的集成

7.什么对我们不起作用

1.伪标签

2.训练时模拟CV

8.期待其他kaggler的无私分享！